

# Предсказание возраста по фото лица

## Отчет по вехе 5

Баранцов Данил, Костырин Николай

4 мая 2026 г.

## 1 Введение

Цели ГП-5:

- Дистилляция знаний: ResNet50 (teacher)  $\rightarrow$  ResNet18 (student)
- Квантизация: динамическая int8 + Post-Training Quantization (FX)
- Прунинг: глобальное L1-unstructured с fine-tune после обрезки
- Кэширование: embedding-кэш + prediction-кэш для ускорения повторного инференса

## 2 Дистилляция знаний

### 2.1 Назначение и архитектура

Дистилляция знаний (Knowledge Distillation, KD) — метод передачи знаний от более тяжёлой и точной модели (teacher) к компактной модели (student). Цель: приблизить качество student к teacher при меньших вычислительных затратах.

Архитектура:

- Teacher: ResNet50 (параметров  $\sim 23$  млн, 90 МБ) — более глубокая сеть с большей representational capacity.
- Student: ResNet18 (параметров  $\sim 11$  млн, 43 МБ) — вдвое компактнее, что даёт выигрыш по размеру и скорости инференса.

В процессе дистилляции параметры teacher замораживаются (`requires_grad=False`, `eval()`), а student обучается минимизировать комбинированный loss:

$$L_{\text{total}} = \lambda_{\text{task}} \cdot \text{SmoothL1}(\hat{y}_s, y) + \lambda_{\text{kd}} \cdot L_{\text{KD}}$$

### 2.2 Типы KD-loss

**Response-based** (по умолчанию): MSE между предсказаниями возраста teacher и student:

$$L_{\text{KD}} = \text{MSE}(\hat{y}_t, \hat{y}_s)$$

**Feature-based**: косинусное рассогласование между эмбедингами backbone:

$$L_{\text{KD}} = 1 - \cos(f_t, f_s)$$

При несовпадении размерностей (teacher: 2048, student: 512) добавляется линейный проекционный слой  $f'_t = W f_t + b$ .

## 2.3 Процесс обучения

Стратегия обучения student при дистилляции:

- Этап 1 — тренировка только головы (backbone заморожен): 1–5 эпох с `lr_head`.
- Этап 2 — полный файн-тюн + дистилляция: 2–10 эпох с `lr_finetune`.
- Early stopping по val RMSE (patience = 5 эпох).
- Логирование: `train_task_loss`, `train_kd_loss`, `train_loss`, `val_rmse`, `val_mae`.

## 2.4 Результаты дистилляции

| Модель                         | RMSE↓  | MAE↓   | Acc@5↑ | Acc@10↑ | Размер, МБ |
|--------------------------------|--------|--------|--------|---------|------------|
| Teacher (ResNet50)             | 8.0062 | 5.5980 | 59.5%  | 83.3%   | 89.98      |
| Student w/o teacher (ResNet18) | 7.8355 | 5.6788 | 56.9%  | 82.5%   | 42.71      |
| Distilled student (ResNet18)   | 8.0090 | 5.6931 | 58.5%  | 82.1%   | 42.71      |

Таблица 1: Сравнение teacher, baseline student и distilled student

Дистиллированный student незначительно уступает baseline (RMSE 8.01 vs 7.84) при обучении всего 3 эпохи в QUICK-режиме. При полном обучении ожидается приближение к качеству teacher за счёт переноса признакового пространства. Ключевой выигрыш: размер модели — 42.71 МБ против 89.98 МБ у teacher (−53%).

## 3 Квантизация модели

Квантизация — переход от 32-битных float-операций к операциям пониженной разрядности с целью уменьшения размера модели и ускорения инференса на CPU.

### 3.1 Динамическая квантизация

Применяется к слоям `nn.Linear`: веса конвертируются в `int8` при загрузке, активации квантизируются динамически во время инференса.

Реализация: `torch.ao.quantization.quantize_dynamic`.

- Не требует калибровочных данных — применяется мгновенно.
- Conv2d-слои остаются в float32 (torch dynamic quant поддерживает только Linear).
- Результат: RMSE = 8.0127, MAE = 5.6963 — практически без деградации качества.

### 3.2 Post-Training Quantization (PTQ)

PTQ через FX Graph Mode (`torch.ao.quantization`):

- Шаг 1: `prepare_fx` — вставка наблюдателей активаций.
- Шаг 2: `calibrate` — прогон 200 изображений из train split (не val!) для сбора статистик диапазонов активаций.

- Шаг 3: `convert_fx` — конвертация в quantized-формат (int8, backend=qnnpack).

Квантизованные модели запускаются только на CPU — torch quantized ops не поддерживают MPS/CUDA.

### 3.3 Результаты квантизации

| Метрика     | Dynamic quant | PTQ int8 | Float32 (baseline) |
|-------------|---------------|----------|--------------------|
| RMSE↓       | 8.0127        | 8.1147   | 8.0090             |
| MAE↓        | 5.6963        | 5.7711   | 5.6931             |
| Размер, МБ↓ | 42.71         | 10.70    | 42.71              |
| Latency, мс | 7.15          | 9.76     | 7.13               |

Таблица 2: Метрики квантизованных моделей (CPU inference)

PTQ даёт ~4-кратное сжатие: 42.71 МБ → 10.70 МБ при деградации RMSE лишь на 0.11 (1.4%). Latency незначительно возрастает — quantized ops в PyTorch FX пока не используют полный потенциал CPU-ускорения на Apple Silicon (qnnpack оптимизирован для ARM, но MPS не задействован).

## 4 Прунинг (обрезка) весов

### 4.1 Метод и параметры

Реализован глобальный L1-unstructured magnitude pruning через `torch.nn.utils.prune.global_unstructured`:

- Цель: обнуление наименьших по абсолютному значению весов.
- Метод: L1Unstructured, применяется глобально ко всем Conv2d и Linear слоям.
- Доля обнуляемых весов (amount): 50%.
- После прунинга маски фиксируются (`remove_pruning`) и модель дообучается (`train_two_stage`, 1 эпоха файн-тюна).

### 4.2 Результаты прунинга

| Этап                    | Разреженность | RMSE↓  | MAE↓   |
|-------------------------|---------------|--------|--------|
| До прунинга (distilled) | 0.0%          | 8.0090 | 5.6931 |
| После прунинга (до FT)  | 50.0%         | 7.9246 | —      |
| После fine-tune (1 эп.) | 50.0%         | 7.8539 | 5.6556 |

Таблица 3: Динамика RMSE при прунинге

50%-прунинг практически не ухудшил качество (RMSE до: 7.92  $\rightarrow$  после FT: 7.85). Важно, что итоговый RMSE после fine-tune (7.8539) даже лучше исходного distilled student (8.0090) — fine-tune фактически провёл дополнительную оптимизацию. Размер state\_dict остаётся 42.71 МБ, так как разреженность хранится в масках; реальное сжатие требует sparse tensor форматов или структурированного прунинга.

## 5 Кэширование инференса

### 5.1 Архитектура кэша

CachedAgePredictor реализует двухуровневый кэш поверх любой модели предсказания возраста:

- PredictionCache: hash(изображение)  $\rightarrow$  предсказанный возраст. Попадание = полный пропуск forward-pass.
- EmbeddingCache: hash(изображение)  $\rightarrow$  эмбединг backbone. Попадание = только проход через голову (Dropout+Linear).
- Full forward: backbone + голова — если оба кэша промахнулись.

Хэширование изображений: MD5 по байтам PIL-изображения или тензора (`detach().cpu()` для корректной работы на GPU/MPS). Размер кэша ограничен (`max_cache_size=1000`). Модель переводится в `eval()` при инициализации предиктора — это критично: активный Dropout давал бы разные предсказания для одного изображения и лишал кэш смысла.

### 5.2 Результаты кэширования

| Проход    | Время (мс) | Время/изображение (мс) | Hit-rate (pred) | Hit-rate (emb) |
|-----------|------------|------------------------|-----------------|----------------|
| Cold      | 1002.3     | 20.05                  | 0%              | 0%             |
| Warm      | 619.8      | 12.40                  | 100%            | 0%             |
| Emb-cache | 632.8      | 12.66                  | 0%              | 100%           |

Таблица 4: Результаты кэширования на 50 изображениях (CPU)

Speedup prediction кэша:  $1.6\times$  (1002 мс  $\rightarrow$  620 мс). Embedding кэш даёт аналогичное ускорение (633 мс), избегая прохода через backbone но всё же выполняя проход через голову. В production-сценариях с высокой долей повторных запросов (e.g., видеопоток одного человека) кэш даёт линейное ускорение:  $k$  повторений  $\rightarrow k$ -кратное снижение средней latency.

## 6 Итоговое сравнение

### 6.1 Качество

| Модель                    | RMSE↓  | MAE↓   | Acc@5↑ | Acc@10↑ | Размер МБ |
|---------------------------|--------|--------|--------|---------|-----------|
| Teacher (ResNet50)        | 8.0062 | 5.5980 | 59.5%  | 83.3%   | 89.98     |
| Student w/o teacher       | 7.8355 | 5.6788 | 56.9%  | 82.5%   | 42.71     |
| Distilled student         | 8.0090 | 5.6931 | 58.5%  | 82.1%   | 42.71     |
| Pruned + fine-tuned (50%) | 7.8539 | 5.6556 | 57.7%  | 83.0%   | 42.71     |

Таблица 5: Сравнение качества всех вариантов модели (val, QUICK-режим)

### 6.2 Метрики деплоя

| Вариант           | Размер МБ↓ | Latency мс↓ | Throughput img/s↑ | Peak RAM МБ↓ |
|-------------------|------------|-------------|-------------------|--------------|
| Distilled float32 | 42.71      | 7.13        | 140.3             | 0.05         |
| Dynamic quant     | 42.71      | 7.15        | 139.8             | 0.00         |
| PTQ int8          | 10.70      | 9.76        | 102.4             | 0.00         |
| Pruned float32    | 42.71      | 6.85        | 145.9             | 0.00         |

Таблица 6: Deployment-метрики (CPU, batch=1, 200 итераций)

## 7 Выводы

В рамках задачи GP-5 реализованы и проверены на реальных данных четыре техники сжатия модели предсказания возраста:

- **Дистилляция (ResNet50→ResNet18):** student-модель сохраняет сопоставимое качество (RMSE 8.01) при вдвое меньшем размере (43 МБ vs 90 МБ). Response-based KD-loss (MSE на предсказаниях) показал стабильное обучение.
- **Динамическая квантизация:** нулевая деградация качества ( $\Delta\text{RMSE} \approx 0.004$ ) без каких-либо калибровочных данных. Практически без изменения latency (влияние на Linear-слои минимально относительно Conv2d).
- **PTQ int8 (FX):** 4-кратное сжатие размера (43 МБ → 11 МБ), деградация RMSE лишь +0.11 (1.4%). Latency незначительно возросла — специфика qnnpack на CPU.
- **Прунинг 50% + fine-tune:** разреженность 50% с восстановлением качества до RMSE 7.85 после 1 эпохи файн-тюна. Итоговое качество лучше исходного distilled student.
- **Кэширование:** 1.6× ускорение на повторных запросах. Embedding-кэш позволяет избежать затратного backbone при повторном инференсе без потери точности.

Итог: наилучший баланс качество/размер достигается комбинацией PTQ (10.7 МБ, RMSE 8.11) или прунинга + FT (42.7 МБ, RMSE 7.85). Кэширование ортогонально остальным методам и совместимо с любым вариантом модели.